

Chapter 1

Explainability in Reinforcement Learning

Today, the majority of Artificial Intelligence (AI) agents with which humans interact are deployed in controlled or low-risk environments. For example, agents capable of playing games such as Go operate in settings where potential errors remain limited in terms of consequences. Even more autonomous systems, such as agents based on large language models, remain constrained by human supervision mechanisms and are often restricted to tasks such as code generation or information retrieval. Safety-critical applications integrating machine learning components are still rare. Introducing autonomous agents in such environments, requires the establishment of strong guarantees. These requirements include in particular:

- validating the system’s functioning before deployment,
- identifying the origin of a malfunction in case of failure,
- and certifying that an observed failure will not occur again.

The main challenge lies in the fact that the performing agents currently available rely on both Reinforcement Learning (RL) and Deep Learning (DL). As a result, the mechanisms underlying their decisions cannot be understood directly from the learned model. This lack of explainability makes their use difficult in safety-critical contexts. To address this limitation, a research field has emerged: eXplainable Reinforcement Learning (XRL), a subfield of eXplainable Artificial Intelligence (XAI). The objective of XRL is to develop explainability methods that make it possible to obtain explanations of RL agents.

This chapter presents the main existing XRL methods. We first define what is meant by an explanation in the context of XAI. We then propose a taxonomy for organizing existing XRL methods. This taxonomy serves as the basis for the state-of-the-art review presented in the remainder of the chapter.

1.1 Explanation in Artificial Intelligence

In this section, we define what an explanation is in the context of XAI and how it can be obtained. We first introduce the concept of explanation in a general.

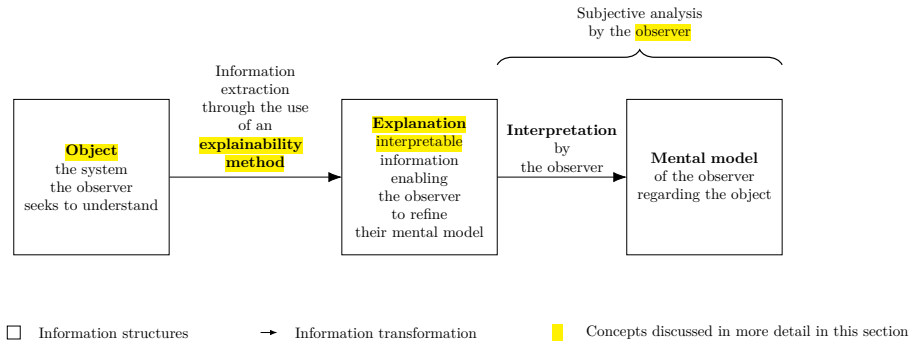


Figure 1.1: Explanation process in explainable artificial intelligence.

We then describe how this concept applies to XAI. Finally, we introduce the concept of explainability methods and describe the properties that characterize explanations in the context of XAI.

1.1.1 Explanation

Establishing what is meant by an explanation is challenging due to the fundamentally multidisciplinary nature of the concept. When we refer to explanation, we are inherently referring to an explanation provided to a human. As soon as humans are involved, the problem is no longer purely computational or mathematical. It involves several fields of research, such as philosophy, psychology, and sociology. So far, these disciplines have not converged on a unified definition of explanation that can be directly applied to XAI.

For this reason, we propose the following definition of explanation. An explanation involves two entities: the object (Definition 1.1.1) being explained and the receiver of the explanation. The receiver is a human who aims to understand how the object operates and will be referred to as the observer throughout this manuscript (Definition 1.1.2). The observer seeks to understand the object and, in doing so, builds a mental model of its functioning. An explanation is therefore what allows the observer to refine and correct their mental model of how the object operates (Definition 1.1.4).

For an observer to understand an explanation, the information it contains must be interpretable (Definition 1.1.3). Information is considered interpretable when it can be directly understood by a human without requiring any additional transformation process.

Definition 1.1.1: Object

The object of an explanation is the system is to be understood by an observer.

Definition 1.1.2: Observer

The observer is the human who aims to understand a object. The observer possesses a mental model of the object being explained. This mental model is an internal representation of how the observer believes the system operates. This representation may be incomplete or inaccurate.

Definition 1.1.3: Interpretable

Information is interpretable when it can be directly understood by an observer without requiring any additional transformation process.

Definition 1.1.4: Explanation

An explanation is any interpretable information that enables an observer to refine or correct their mental model of the object of the explanation.

This definition of explanation provides the foundation for the approach to explainability adopted throughout this manuscript. Its generality allow it to encompass the different explainability methods considered in the following sections.

Moreover, it avoids a binary interpretation of explanation, where a system is considered either explainable or not explainable. From our perspective, the notion of non-explainable system is not meaningful. Considering a system as impossible to explain would therefore contradict the general objective of scientific understanding.

This quantitative perspective allows us to express an intuitive idea: not all explanations provide the same level of value. The effectiveness of an explanation depends on its ability to improve the observer's understanding of the system. For example, describing a marginal aspect of a system is different from correcting an observer's misunderstanding of a key mechanism.

Figure 1.1 summarizes this whole explanation process, from the object of the explanation to the mental model refined by the observer.

1.1.2 Explainable Artificial Intelligence

Now that the general framework of explanation has been established, we can examine how this concept applies in the context of AI. In the context of XAI, the object of explanation is an AI system. More precisely, XAI emerged as a response to the success of DL models. These models achieve high performance through connectionist architectures based on deep Neural Networks (NNs). In these architectures, information is represented in a distributed manner across the network through patterns of activation and connections between computational units. Unlike symbolic approaches, individual components of NNs cannot be directly associated with human-defined concepts. From this starting point, we identify two types of approaches:

- **Substitutive explainability:** In this approach, the underlying assumption is that there is nothing to explain within a NN. There is no semantic meaning associated with each computational component, and therefore nothing to interpret. This corresponds to the so-called “black box” view

of NNs. Under this assumption, XAI consists of replacing as much of the DL system as possible with alternative AI methods.

- **Intrinsic explainability:** In this approach, it is argued that NNs inherently contain mechanisms that can be explained. The limitation is considered to lie not in the networks themselves, but in the absence of adequate tools to analyze their functioning.

This manuscript considers methods from both approaches in order to provide a comprehensive overview of the XAI field. According to the definition of explanation adopted in this manuscript (Definition 1.1.4), we consider that any system can be explained. As a consequence, the perspective adopted throughout this manuscript is that of intrinsic explainability.

1.1.3 Explainability Method in XAI

Now that the general framework of XAI has been established, we need to further detail the properties of an explanation in this context. In our framework, an explanation in the field of XAI can be characterized by three components:

- **Source:** refers to the origin of the information used by the explainability method. The explainability method transforms the raw information produced by the AI system into an explanation that can be understood by an observer.
- **Form:** refers to the representation through which the explanation is communicated. The form defines how the information is presented to the observer and must allow human interpretation. An explanation can take many different forms, including text, images, computer code, and mathematical equations.
- **Subject:** refers to the aspect of the AI system that is being explained. In XAI, this distinction is commonly associated with the difference between local and global explanations. A local explanation describes why an AI system produced a specific output in a given situation. A global explanation aims to describe the general functioning of the AI system.

These three aspects of an explanation are entirely determined by the process used to produce it. We refer to this process as an explainability method. An explainability method is a transformation process that maps a source of information into an explanation characterized by a specific form and subject. Research in XAI focuses on developing new methods capable of performing this transformation in order to overcome the lack of interpretability associated with DL models.

1.2 Taxonomy of XRL Methods

Now that the concept of XAI has been defined, we turn to its application in RL, known as XRL. As in XAI, explanations in XRL are produced by explainability methods and can be described in terms of their subject, source, and form. This section reviews the possible variations of these three aspects in the specific context of RL.

1.2.1 Subject

The first question to address is the subject of explanation in the context of RL. To answer this question, it is necessary to identify the different components involved in a RL system and determine which of them constitutes the object of explanation. A RL system can be described through two main components: the environment and the policy.

The environment defines the dynamics in which the agent evolves, including the possible states, observations, actions, and transitions. Environments are simulations designed by humans, and their dynamics are explicitly defined. Therefore, it is assumed that the environment is known to the observer and does not require explanation.

The main object of interest is therefore the agent’s policy. The policy represents the decision-making mechanism of the agent. It is a function that maps observations to actions and is learned through interaction with the environment in order to maximize the expected cumulative reward. The policy is the component that encodes the learning process and that XRL aims to explain.

Explaining a policy introduces additional challenges compared with classical XAI settings. In traditional XAI problems, the objective is to explain an isolated decision, such as determining which features of an image led to a classification result. In contrast, RL introduces a temporal dimension into the decision-making process. An action is not only determined by the current observation but also influences future states and rewards. Therefore, understanding the behavior of an agent may require considering a sequence of interactions between the agent and its environment.

This temporal dimension leads to a distinction between two objectives of explanation in XRL:

- **Decision explanations:** focus on identifying the elements of the current observation that influenced the action selected by the policy. This type of explanation is close to classical XAI approaches, as it analyzes the relationship between inputs and outputs while abstracting away the sequential nature of the decision process.
- **Strategy explanations:** aim to characterize the agent’s behavior over time. They focus on sequences of decisions and interactions with the environment in order to understand how the agent achieves its long-term objective. This type of explanation is specific to RL, as it addresses the temporal and goal-oriented nature of the learning process.

These two objectives should not be considered as strictly separated categories, but rather as two extremes of a continuum. Indeed, understanding the elements of an observation that influence a specific decision can help the observer generalize this information to infer broader aspects of the agent’s behavior. Conversely, understanding the agent’s overall strategy can provide insights into the reasoning behind individual decisions.

1.2.2 Source

We identify three sources of explanation in RL, all obtained during or after the learning process and containing information about the agent:

- 177 • **Policies:** policies are functions that generate a distribution over the action
178 space for a given observation. The objective of RL is to learn a policy that
179 maximizes the expected cumulative future rewards. During the learning
180 phase, the policy is iteratively improved through interactions with the
181 environment.
- 182 • **Trajectories:** trajectories represent sequences of successive interactions
183 between the agent and the environment. At each step, the agent selects
184 an action based on its observation, which modifies the environment and
185 produces a reward as well as a new observation. This process continues
186 until the end of the trajectory.
- 187 • **Value functions:** value functions provide predictions about the future
188 outcome of a trajectory from a given observation. The most commonly
189 used value function is the critic function, which estimates the expected
190 sum of future rewards. This prediction guides the learning process, as the
191 agent updates its policy to maximize the estimated future return. However,
192 other types of value functions can also be used as sources of explainability.

193 1.2.3 Form

194 We identify six forms of explanation in the RL setting:

- 195 • **Policy Model:** an interpretable model of the agent’s policy function.
- 196 • **Saliency Map:** a weighting of the input features representing their
197 influence on the output of the policy function.
- 198 • **Counterfactual:** a modified observation obtained by perturbing the
199 original observation in order to produce a different action from the agent.
- 200 • **Agent Trajectory:** a sequence of successive interactions between the
201 agent and the environment.
- 202 • **Trajectory Prediction:** a prediction about the future evolution of the
203 trajectory produced by a value function.
- 204 • **Interaction Model:** an interpretable model of the interactions between
205 the agent and the environment.

206 Among the three components characterizing an explanation, the form is the
207 one that varies most significantly across explainability methods. While subjects
208 and sources tend to be relatively stable in the XRL literature, the diversity in
209 how explanations are presented is considerably greater. The list above is by no
210 means definitive and may evolve as new approaches emerge. For this reason, the
211 following section is organized according to the form of explanations and presents
212 existing explainability methods based on this criterion.

213 1.3 State-of-the-Art Review of XRL Methods

214 The purpose of this section is to detail the functioning of existing explainability
215 methods (see Table 1.1). They are presented according to the form of explanation
216 they produce, following the taxonomy introduced in Section 1.2.

Explanation Subject	Explanation Form	Explainability Method Family	Source of Explanation		
			Policy	Trajectories	Value functions
Decision	Policy Model	<i>Interpretable policy learning</i>	✓		
		<i>Policy approximation via interpretable model</i>	✓		
	Saliency Map	<i>Observation perturbation</i>	✓		
		<i>Observation gradient analysis</i>	✓		
		<i>Attention-based architecture</i>	✓		
	Counterfactual	<i>Latent space usage for counterfactual generation</i>	✓		
Strategy	Agent Trajectory	<i>Hierarchical decomposition</i>	✓	✓	
		<i>Extraction via critic analysis</i>		✓	✓
	Trajectory Prediction	<i>Critic enrichment</i>			✓
	Interaction Model	<i>Bayesian network learning</i>		✓	
		<i>Latent space usage for MDP generation</i>	✓		

Table 1.1: Taxonomy of explainability methods in reinforcement learning.

Each subsection introduces one of the six forms of explanation identified in Section 1.2.3. Within each subsection, each subsubsection is dedicated to a family of methods capable of producing that form of explanation.

A family of methods is defined as a set of methods that share the same source, form, and subject of explanation, as well as a similar procedure for generating explanations. Methods belonging to the same family may differ in their implementation or in the details of the generation process, while relying on the same underlying principle.

1.3.1 Policy Model

The policy function can be represented by a wide variety of function classes. In modern RL, NNs is the most commonly used models for representing policies. Although these models achieve the highest performance, their connectionist nature makes their internal representations non-interpretable.

For this reason, a substitutive approach to XRL aims to replace these models with alternative models considered interpretable. Such models can be understood through direct analysis of their representations. The main classes of interpretable models used to represent policy functions include decision trees [Topin et al., 2021], algorithms [Trivedi et al., 2022], formal logic [Payani and Fekri, 2019], and linear models [Garreau and von Luxburg, 2020].

Explanations derived from an interpretable model primarily provide insights into the decision-making process, since what is modeled is the mapping between an observation and an action. If the model is sufficiently simple or well structured, the observer can form a mental representation of a sequence of decisions across the environment and thus obtain information about the agent’s strategy.

The main limitation of these methods is that interpretable models rely on symbolic representations. When the policy to be represented is too complex, achieving comparable performance requires a large number of rules or condi-

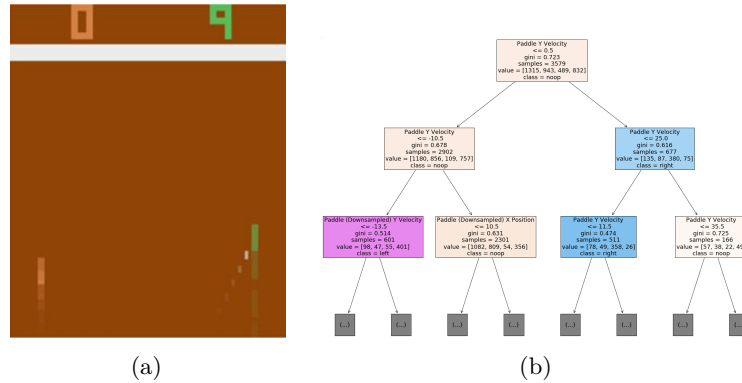


Figure 1.2: Policy approximation via an interpretable model [Sieusahai and Guzdial, 2021]. (a) shows an observation from the Atari game “Pong”, and (b) shows the decision tree obtained by approximating a deep reinforcement learning policy trained on this environment.

tions. The resulting model can quickly become difficult to analyze and lose its interpretability.

For example, representing a policy capable of playing Go using a symbolic model would require considering a very large number of possible situations and specific cases. Even if such a representation were possible, the resulting model would likely become intractable for human analysis due to its complexity. This illustrates why NN-based approaches have become dominant in such domains, as they can learn complex decision strategies without requiring an explicit enumeration of rules.

To obtain an interpretable policy model, two explainability methods are available: *interpretable policy learning* and *policy approximation via interpretable model*.

Interpretable Policy Learning

This method aims at learning a policy represented by an **interpretable** model [Topin et al., 2021, Trivedi et al., 2022]. It is the most direct approach to addressing the challenges posed by XRL. This type of training requires modifying the Markov Decision Processes (MDP) in order to adapt it to the search space and to the type of policy representation. Moreover, this search space often needs to be reduced to make learning feasible. Such a **reduction** requires the integration of **expert knowledge** into the learning process. With this method, the *interpretable policy function* itself is used to provide explanations to the observer.

Policy Approximation via Interpretable Model

An interpretable model can be trained to approximate a policy learned through deep RL [Sieusahai and Guzdial, 2021]. In this setting, the problem is formulated as a supervised learning task, where the original deep RL policy is used to label each observation with the action it selects (see Figure 1.2a). Using this dataset, the interpretable model is trained to reproduce the behavior of the original policy,

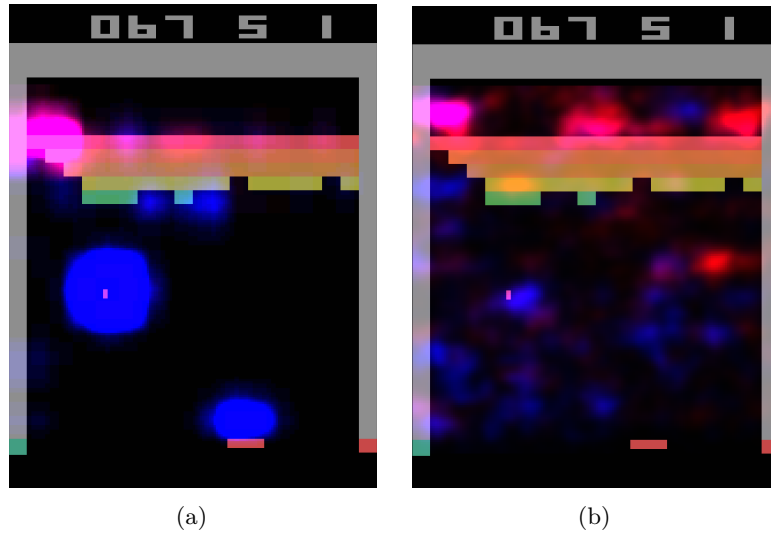


Figure 1.3: Heatmaps obtained by [Greydanus et al., 2018] for the same observation of the Atari game “Breakout”. Highlighted areas correspond to the most influential regions for the policy function’s decision. (a) is obtained via observation perturbation (see Section 1.3.2), whereas (b) is obtained via observation gradient analysis (see Section 1.3.2).

yielding for instance the decision tree illustrated in Figure 1.2b. The resulting interpretable model is then used as an explanation for the observer. If this approximation is sufficiently faithful, it can also be deployed in the environment in place of the original policy, thereby providing a high degree of control over the agent.

1.3.2 Saliency Map

The influence of each part of the input on the action selected by the policy can be visualized using heatmaps (Figure 1.3). In the RL setting, the input corresponds to the observation received by the agent at a given time step. Heatmaps therefore highlight the regions of the observation that have the greatest influence on the action selection process. By providing this information, they allow the observer to identify which elements of the environment are considered important by the policy when making a decision.

A limitation of heatmap-based methods is that the resulting explanations are not always directly meaningful to the observer. As illustrated in Figure 1.3b, interpreting which regions are important often requires human reasoning and involves a degree of subjectivity. This limitation becomes particularly relevant in RL, where the objective is not only to explain a single decision but also to understand the factors influencing a sequence of decisions over time.

Three explainability methods can be used to obtain this type of explanation: *observation perturbation*, *observation gradient analysis*, and *attention-based architectures*.

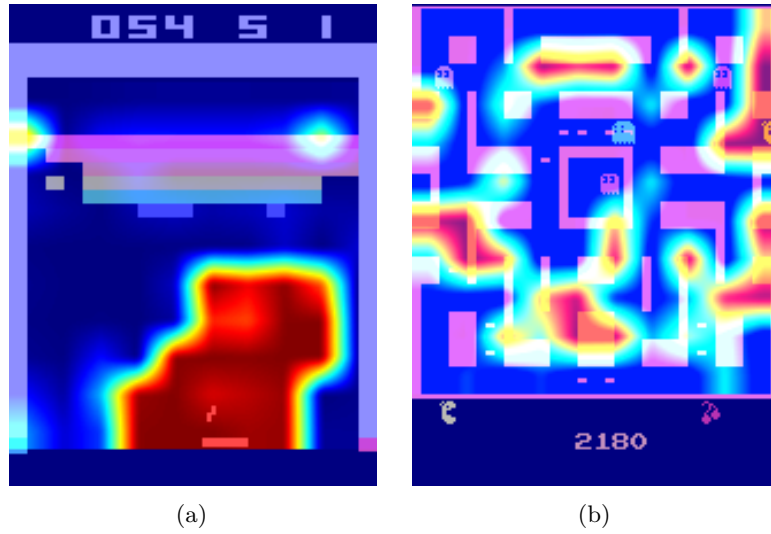


Figure 1.4: Attention-based heatmaps obtained by [Mott et al., 2019] for the Atari games “Breakout” and “Ms. Pac-Man” (see Section 1.3.2). In (a), the highlighted regions are easily interpretable, whereas in (b), the attention is spread across many regions of the observation, making the explanation much harder to interpret.

294 *Observation Perturbation*

295 Perturbation analysis consists in applying various **modifications** to the obser-
 296 vation in order to study their **impact** on the output of the **policy function**
 297 [Greydanus et al., 2018]. The greater the variation in the output caused by a
 298 perturbation applied to a given component of the input vector, the higher the
 299 **weight** assigned to that component. The magnitude of the perturbation is
 300 measured by computing the distance between the original output and the output
 301 obtained from the perturbed input. Using this set of weights, a *heatmap* is
 302 constructed, enabling the observer to visualize which parts of the observation
 303 vector are most sensitive in the action-selection process (see Figure 1.3a).

304 *Observation Gradient Analysis*

305 Observation **gradient** analysis refers to a set of methods that require a differen-
 306 tiable policy function in order to extract information from it [Simonyan et al., 2014,
 307 Weitkamp et al., 2019, Huber et al., 2019]. To this end, the gradient of the pol-
 308 icy function is computed with respect to all **components of the input vector**.
 309 For each input component, the resulting gradient provides a weighting that
 310 represents its importance in the output of the policy function. Based on these
 311 weightings, a *heatmap* is constructed to allow the observer to visually identify
 312 the parts of the input vector that are the most influential in the policy function’s
 313 output (see Figure 1.3b).

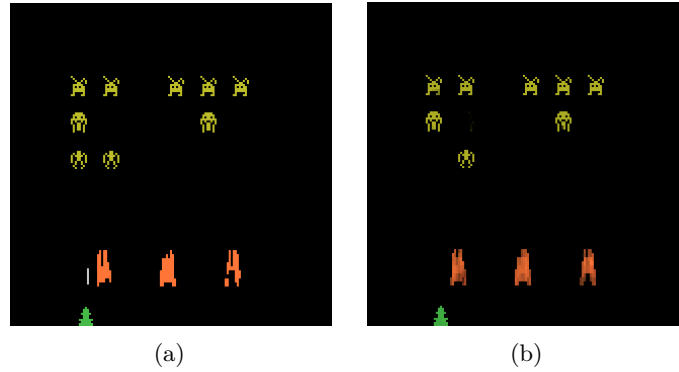


Figure 1.5: Counterfactual explanation of the agent’s behavior for the game *Space Invaders* [Olson et al., 2021]. (a) is the original observation, for which the agent selects the action “LEFT”, whereas (b) is the counterfactual observation, for which the agent selects the action “RIGHT_FIRE”.

314 *Attention-Based Architecture*

315 **Attention** blocks constitute a NN architecture that makes it possible to weight
 316 the relative importance of one part of a vector with respect to the other parts
 317 of the same vector [Vaswani et al., 2023]. These weights are computed as a
 318 function of the vector itself and are learned by the NN during training. For
 319 a given observation, each component of the observation vector is assigned a
 320 scalar value. It can therefore be assumed that the components associated
 321 with the highest weights are the most influential in determining the output
 322 [Mott et al., 2019, Itaya et al., 2021]. This attention-based *heatmap* is then
 323 provided to the observer as the explanation (see Figure 1.4a). However, as
 324 shown in Figure 1.4b, the attention weights are not always concentrated on a
 325 small, easily interpretable set of regions, which can make this type of explanation
 326 difficult to exploit.

327 1.3.3 Counterfactual

328 In the context of RL, counterfactual explanations are used to generate new
 329 observations. This new observation, distinct from the original one, leads the
 330 agent to adopt an alternative behavior. A counterfactual explanation aims to
 331 explain the agent’s decision-making process. Modifying the observation informs
 332 the observer about the adjustments required for the agent to adopt a different
 333 behavior. The method available to generate counterfactuals is the *use of the*
 334 *Latent Space (LS) for counterfactual generation*.

335 *Latent Space Usage for Counterfactual Generation*

336 In this method, **generative models** are used [Bond-Taylor et al., 2021] to
 337 exploit the Latent Representation (LR). This type of model learns to generate
 338 new data by imitating the training data distribution. In this context, generative
 339 models aim to produce alternative observations that are likely to alter the agent’s
 340 actions (see Figure 1.5) [Olson et al., 2021].

341 The main challenge of this approach lies in the fact that, to serve as explana-
 342 tions, counterfactuals must be both **close** and **feasible**. A close counterfactual
 343 refers to one that does not differ significantly from the original observation. A
 344 feasible counterfactual corresponds to an observation that is meaningful within
 345 the considered environment.

346 1.3.4 Agent Trajectory

347 Until now, we have presented methods that focus on explaining individual
 348 decisions made by the agent. We now consider methods that aim to provide
 349 insight into the agent’s strategy by analyzing its behavior over time.

350 The most direct approach consists in visualizing agent trajectories, most
 351 commonly through complete episodes. By analyzing these trajectories, the
 352 observer can infer the dynamics underlying the agent’s behavior and how its
 353 decisions evolve through time. This type of visualization is not specific to XRL.
 354 In practice, visualizing agent episodes is a standard step in the development and
 355 evaluation of RL. It provides a simple way to verify that the agent behaves as
 356 expected and to detect potential implementation errors.

357 This simple approach can be further enhanced using methods such as *hierar-*
 358 *chical decomposition* and *extraction via critic analysis*.

359 *Extraction via Critic Analysis*

360 In this method, the predictions of the critic are used to determine which **trajec-**
 361 **tories to present to the observer** [Sequeira and Gervasio, 2020]. A set of
 362 states is selected from multiple trajectories. For a given state, all possible actions
 363 are examined. The state is assigned a value equal to the difference between
 364 the critic’s prediction for the lowest-valued action and the prediction for the
 365 highest-valued action. Trajectories containing the states with the highest values
 366 according to this procedure are then chosen to be presented to the observer.
 367 These trajectories are considered **critical moments** of the agent and are there-
 368 fore relevant for analysis. For this method, the *selected trajectories* represent the
 369 interpretable information provided to the observer.

370 *Hierarchical Decomposition*

371 Hierarchical agents are composed of a set of **sub-policies** specialized for specific
 372 tasks within subsets of the environment’s states. This method distributes the
 373 optimization of the reward function across multiple models, significantly simpli-
 374 fying the learning process. Moreover, this **decomposition** of the policy function
 375 can also provide explainability in the agent’s strategy [Beyret et al., 2019]. If
 376 a sub-policy is selected, it indicates that the agent will execute a specialized
 377 behavior. This behavioral specialization provides insight into the strategy the
 378 agent employs to maximize the reward function. The agent’s *trajectory*, together
 379 with the *sub-policy* selected for each action, is used as interpretable information.

380 1.3.5 Trajectory Prediction

381 Making predictions about the future of a trajectory allows one to form a repre-
 382 sentation of what the system expects, based on its training experience. These

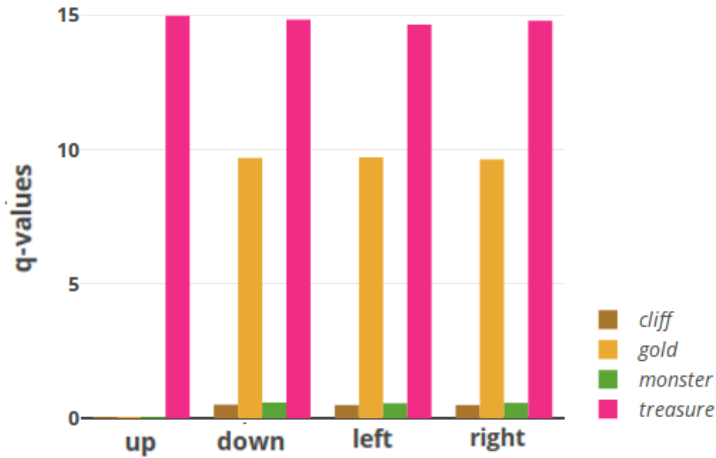


Figure 1.6: Prediction of sub-rewards as a function of the action selected by the agent for a given state [Juozapaitis et al., 2019]. Depending on the action chosen by the agent, one can determine which sub-reward it aims to maximize.

predictions are produced using value functions. Such functions provide insights into the future evolution of the agent’s trajectory. By their nature, this form of explanation informs the observer about the strategy adopted by the agent. Value functions provide an estimate of the expected cumulative reward from a given state. Consequently, a value function maps an observation to a scalar value in $\mathbb{R}$.

This aggregated prediction provides limited information about the underlying factors that contribute to this value. It does not directly reveal which aspects of the task are being optimized or how the predicted return is composed. To improve the interpretability of value-based explanations, a technique known as *critic enrichment* can be employed.

Critic Enrichment

In order to obtain interpretable information from the predictions of the **critic**, it is possible to **decompose** the critic into multiple **sub-functions** [Juozapaitis et al., 2019]. The reward function is thus decomposed into multiple sub-reward functions, each representing a **sub-objective** of the task to be accomplished. Similarly, for each of these sub-reward functions, corresponding sub-critic functions are defined, along with a function that combines their outputs. During training, the agent selects its actions so as to maximize this combination function of the sub-critics. Using this approach, for each observation and action, a prediction is obtained regarding the sub-objectives the agent is attempting to maximize (see Figure 1.6). This *prediction* is then used as the explanation for the observer.

1.3.6 Interaction Model

Modeling the interactions between an agent and its environment makes it possible to understand how the agent’s decisions are made and what impact they have on the environment. By incorporating these interactions into an explanatory

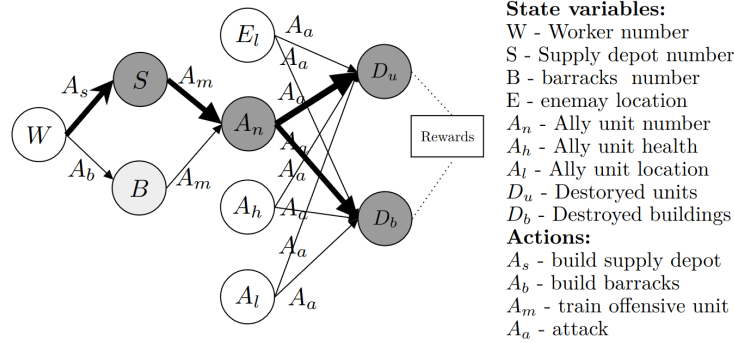


Figure 1.7: Agent-environment interaction model for the video game *StarCraft II* [Madumal et al., 2020]. Nodes represent random variables, and edges represent causal relationships between them.

model, one can better grasp the complex relationships between the agent and its environment, thereby identifying the determining factors in the decision-making process. By explicitly handling the agent-environment interaction, this type of explanation aims to explain the agent's strategy. The methods used to obtain explanations in the form of agent-environment interactions are: *Bayesian network learning* and *LS usage for MDP generation*.

Bayesian Network Learning

The evolution of an agent within an environment can be observed as a set of **random variables** connected through a **Bayesian network** [Madumal et al., 2020]. Random variables are defined to represent both external aspects (the environment) and internal aspects of the agent (the actions). By analyzing trajectories, it becomes possible to infer causal relationships among these random variables. At the end of this process, a weighted causal graph is obtained, which serves as an explanation for the observer (see Figure 1.7). This *Bayesian network*, representing the interactions between the agent and the environment, is then used as the explanation.

Latent Space Usage for MDP Generation

Each layer of a NN can be viewed as a projection from one space to another [Zahavy et al., 2017]. As information propagates through successive layers of a NN, it becomes increasingly abstract. It can therefore be assumed that two vectors that are close in this projected space are also close in terms of the agent's perception. Based on this observation, it is possible to **redefine an MDP** by leveraging the **abstract representations** produced by the final layers of a NN. Once the new MDP has been constructed, a model of the interaction between the agent and its environment is obtained. This model, represented as a *graph*, is then used as the explanation.

Figure 1.8 illustrates this idea using a two-dimensional *t-SNE* projection of the LS of the NN used to represent the policy. Each point is colored according to the value function's prediction for the corresponding state. An agreement

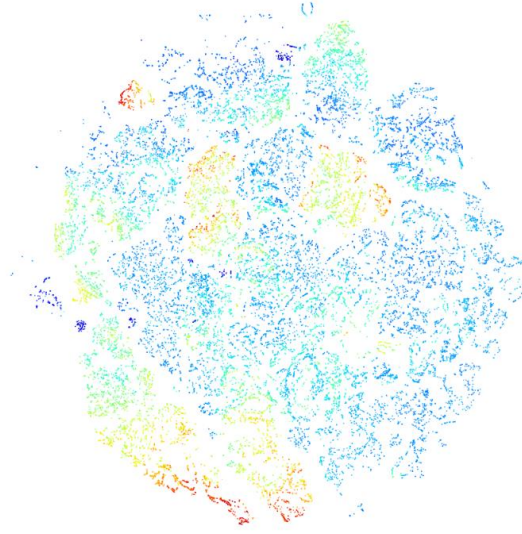


Figure 1.8: t -SNE projection of a neural network's latent space [Zahavy et al., 2017]. Each point corresponds to a state, and its color represents the value function's prediction for that state, with warmer colors indicating higher predicted values.

438 can be observed between the structure of this projected space and the critic's
 439 predictions: states that are assigned similar values by the critic also tend to be
 440 located close to one another in the projection. This agreement makes it possible
 441 to manually identify clusters of states, which likely correspond to states that are
 442 perceived as similar by the agent.

Bibliography

- [Beyret et al., 2019] Beyret, B., Shafti, A., and Faisal, A. A. (2019). Dot-to-dot: Explainable hierarchical reinforcement learning for robotic manipulation. In 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE. **Article**.
- [Bond-Taylor et al., 2021] Bond-Taylor, S., Leach, A., Long, Y., and Willcocks, C. G. (2021). Deep generative modelling: A comparative review of vaes, gans, normalizing flows, energy-based and autoregressive models. **Article**.
- [Garreau and von Luxburg, 2020] Garreau, D. and von Luxburg, U. (2020). Explaining the explainer: A first theoretical analysis of lime. In Chiappa, S. and Calandra, R., editors, Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics, volume 108 of Proceedings of Machine Learning Research, pages 1287–1296. PMLR. **Article**.
- [Greydanus et al., 2018] Greydanus, S., Koul, A., Dodge, J., and Fern, A. (2018). Visualizing and understanding atari agents. **Article**. **Code**.
- [Huber et al., 2019] Huber, T., Schiller, D., and André, E. (2019). Enhancing explainability of deep reinforcement learning through selective layer-wise relevance propagation. In KI 2019: Advances in Artificial Intelligence: 42nd German Conference on AI, Kassel, Germany, September 23–26, 2019, Proceedings 42, pages 188–202. Springer. **Article**.
- [Itaya et al., 2021] Itaya, H., Hirakawa, T., Yamashita, T., Fujiyoshi, H., and Sugiura, K. (2021). Visual explanation using attention mechanism in actor-critic-based deep reinforcement learning. **Article**.
- [Juozapaitis et al., 2019] Juozapaitis, Z., Koul, A., Fern, A., Erwig, M., and Doshi-Velez, F. (2019). Explainable reinforcement learning via reward decomposition. In IJCAI/ECAI Workshop on explainable artificial intelligence. **Article**.
- [Madumal et al., 2020] Madumal, P., Miller, T., Sonenberg, L., and Vetere, F. (2020). Explainable reinforcement learning through a causal lens. In Proceedings of the AAAI conference on artificial intelligence, volume 34, pages 2493–2500. **Article**.
- [Mott et al., 2019] Mott, A., Zoran, D., Chrzanowski, M., Wierstra, D., and Jimenez Rezende, D. (2019). Towards interpretable reinforcement learning using attention augmented agents. Advances in neural information processing systems, 32. **Article**.

- [Olson et al., 2021] Olson, M. L., Khanna, R., Neal, L., Li, F., and Wong, W.-K. (2021). Counterfactual state explanations for reinforcement learning agents via generative deep learning. *Artificial Intelligence*, 295:103455. **Article. Code.**
- [Payani and Fekri, 2019] Payani, A. and Fekri, F. (2019). Inductive logic programming via differentiable deep neural logic networks. **Article. Code.**
- [Sequeira and Gervasio, 2020] Sequeira, P. and Gervasio, M. (2020). Interestingness elements for explainable reinforcement learning: Understanding agents’ capabilities and limitations. *Artificial Intelligence*, 288:103367. **Article.**
- [Sieusahai and Guzdial, 2021] Sieusahai, A. and Guzdial, M. (2021). Explaining deep reinforcement learning agents in the atari domain through a surrogate model. **Article.**
- [Simonyan et al., 2014] Simonyan, K., Vedaldi, A., and Zisserman, A. (2014). Deep inside convolutional networks: Visualising image classification models and saliency maps. **Article. Code.**
- [Topin et al., 2021] Topin, N., Milani, S., Fang, F., and Veloso, M. (2021). Iterative bounding mdps: Learning interpretable policies via non-interpretable methods. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 9923–9931. **Article.**
- [Trivedi et al., 2022] Trivedi, D., Zhang, J., Sun, S.-H., and Lim, J. J. (2022). Learning to synthesize programs as interpretable and generalizable policies. **Article. Code.**
- [Vaswani et al., 2023] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2023). Attention is all you need. **Article.**
- [Weitkamp et al., 2019] Weitkamp, L., van der Pol, E., and Akata, Z. (2019). Visual rationalizations in deep reinforcement learning for atari games. **Article.**
- [Zahavy et al., 2017] Zahavy, T., Zrihem, N. B., and Mannor, S. (2017). Graying the black box: Understanding dqns. **Article.**